

Question 1

```
public class OPT {

    private int capacity;
    private List<Integer> list = new ArrayList<Integer>();
    private int cnt = 0;
    public OPT(int[] arr,int capacity) {
        this.capacity = capacity;
        for (int i = 0; i < arr.length; i++) {
            if (list.size() < capacity) {
                if (!list.contains(arr[i])){
                    list.add(arr[i]);
                    cnt++;
                }
            } else {
                if(list.contains(arr[i]))
                    continue;
                else{
                    cnt++;
                    int change_index = 0;
                    int max_dis = 0;
                    for(int index = 0; index < capacity; index++){
                        int page = list.get(index);
                        int j;
                        for(j = i+1; j < arr.length; j++)
                            if(arr[j] == page)
                                break;
                        if(j-i > max_dis){
                            max_dis = j-i;
                            change_index = index;
                        }
                    }
                    list.set(change_index,arr[i]);
                }
            }
        }
        System.out.println(capacity + "个页框缺页" + cnt + "次");
    }
}
```

```
public class FIFO {

    private int capacity;
    private int index = 0;
    private Queue<Integer> queue ;
    private int cnt = 0;
    public FIFO(int[] arr,int capacity) {
        queue = new ArrayBlockingQueue<Integer>(capacity);
        this.capacity = capacity;
        for (int i = 0; i < arr.length; i++) {
            if (queue.size() < capacity) {
                if (!queue.contains(arr[i])) {
                    queue.add(arr[i]);
                }
            }
        }
    }
}
```

```

        } else {
            continue;
        }
    } else {
        if (!queue.contains(arr[i])) {
            queue.poll();
            queue.add(arr[i]);
        } else {
            continue;
        }
    }
    cnt++;
}
System.out.println(capacity + "个页框缺页" + cnt + "次");
}
}

```

```

public class LRU {

    // 初始化一个容量为 5 的栈
    private MyStack<Integer> stack ;
    private int index = 0; // 一个数组下标的标识，用来除去栈底元素
    private int capacity;
    public LRU(int[] arr,int capacity) {
        this.capacity = capacity;
        stack = new MyStack<Integer>(capacity);
        for (int i = 0; i < arr.length; i++) { // 将数组里 数据存入栈中
            if (stack.getLength() < stack.size()) { // 判断栈不满
                if (!stack.exist(arr[i])) { // 判断栈中有没有该元素，有的话返回true
                    stack.push(arr[i]);
                    stack.traverse();
                } else { // 如果有该元素，删除掉栈中该元素，并将该元素置于栈顶
                    stack.removeSameElement(arr[i]);
                    stack.push(arr[i]);
                }
            } else { // 栈已满，删除到栈底元素
                if (stack.exist(arr[i])) { // 继续判断如果栈中存在相同元素，将该元素从栈
                    // 中取出放到栈顶
                    stack.removeSameElement(arr[i]);
                    stack.push(arr[i]);
                } else { // 栈中没相同元素，删除栈底元素，在栈顶加新元素
                    stack.remove(arr[index++]);
                    stack.push(arr[i]);
                    stack.traverse(); // 遍历栈中元素
                }
            }
        }
        System.out.println(capacity + "个页框缺页" + stack.getNum() + "次");
    }
}

```

```

public class Test {
    public static void main(String[] args){
        int[] arr = {0, 9, 8, 4, 4, 3, 6, 5, 1, 5, 0, 2, 1, 1, 1, 1, 8, 8, 5,
            3, 9, 8, 9, 9, 6, 1, 8, 4, 6, 4, 3, 7, 1, 3, 2, 9, 8, 6, 2, 9,
            2, 7, 2, 7, 8, 4, 2, 3, 0, 1, 9, 4,

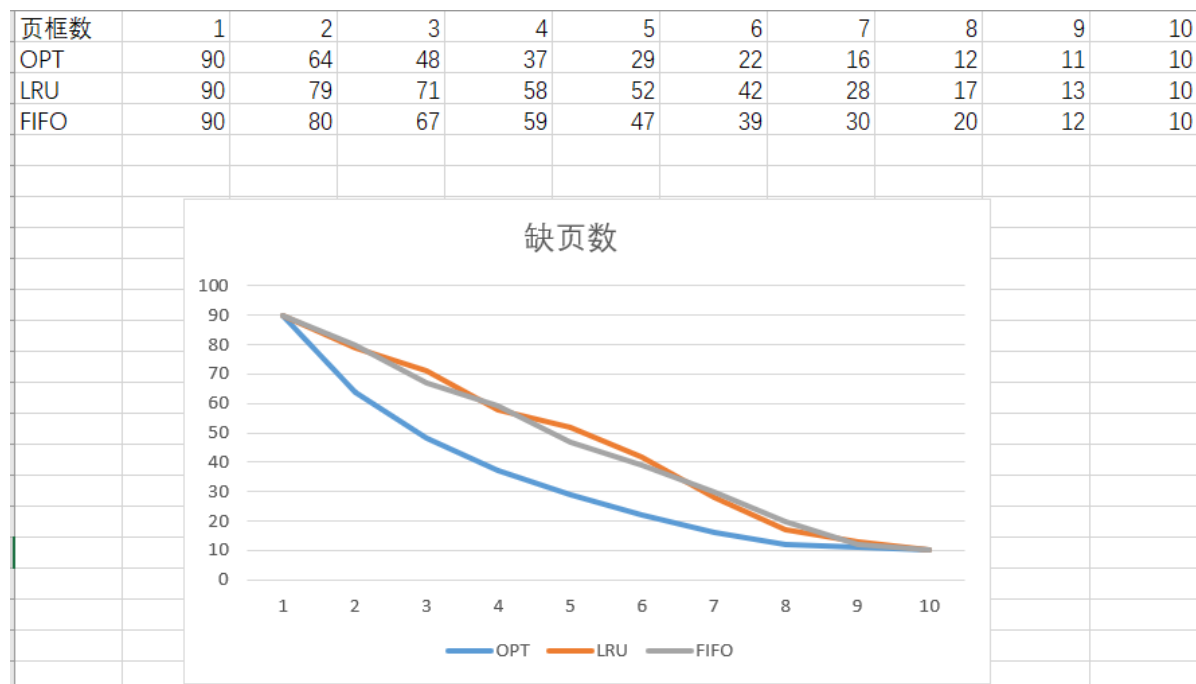
```

```

        7, 1, 5, 9, 1, 7, 3, 4, 3, 7, 1, 0, 3, 5, 9, 9, 4, 9, 6, 1, 7,
5, 9, 4, 9, 7, 3, 6, 7, 7, 4, 5, 3, 5, 3, 1, 5, 6, 1,
        1, 9, 6, 6, 4, 0, 9, 4, 3};
    for (int i = 1; i <= 10; i++) {
        OPT opt = new OPT(arr, i);
    }
    for (int i = 1; i <= 10; i++) {
        FIFO fifo = new FIFO(arr, i);
    }
    for (int i = 1; i <= 10; i++) {
        LRU lru = new LRU(arr, i);
    }
}
}

```

得到结果和绘制图表如下



Question 2

二级页表起始地址为0x80000000，每页大小为4MB，所以对应第0x80000000>>12 = 0x80000个页，每个页表项在页表中占4B空间，所以页目录映射的没存在二级页表中的偏移量为0x80000*4 = 0x200000，所以页目录起始地址为0x80000000 + 0x200000 = 0x80200000

Question 3

1. 32位地址，地址空间为4GB。页内偏移量12位，每页大小为4KB。

2. 访问0x0: 查页目录项0000: 有效标志为0，缺页

访问0x00803004: 化为2进制，得到页目录位0000000010，查页目录项0002: 0x5001，有效位位1，页表物理地址0x5000。再由二级页表索引位0000000011，页表项0003: 0x20001，有效标志为1，页面物理地址为0x20000。页内偏移位000000000100 = 4B，查找到，0001: 0x32_6001系统为大端和小端访问到的数据分别是00000000和00000001

访问0x00402001: 化为2进制，得到页目录位0000000001，查页目录项0001: 0x1001，有效位位1，页表物理地址0x1000。再由二级页表索引位0000000010，页表项0002: 0x50001，有效标志为1，页面物理地址为0x50000。页内偏移位00000000000001 = 1B，查找到，0000: 0x9000系统为大端和小端访问到的数据都是00000000

3. 低12位页内偏移为0x28，物理页框号为0x326000。0x20000处页表的0001偏移处存有该物理页框号且有效位为1，可知虚拟地址的页表偏移位为0000000001。从页目录中查到页目录项0003中页表物理地址为0x20000且标志位为1，可知虚拟地址页目录位为0b0000000011。综上，虚拟地址为0000 0000 1100 0000 0001 0000 0010 1000，即0x00c01028